

Bases de Datos II — Semana 16 Santiago Chavarro Herrera & Juan Pablo Sánchez — Grupo 10

¿Cómo cambió mi perspectiva como ingeniero?

Al comenzar el semestre, nuestra visión de una base de datos era casi instintiva: si el sistema necesitaba guardar datos, la respuesta era PostgreSQL o cualquier motor relacional. Era lo que conocíamos, lo que nos enseñaron primero, y lo que nos parecía "seguro". El modelo relacional tiene tablas, tiene reglas, tiene orden. Eso se sentía correcto.

El proyecto integrador nos obligó a romper esa certeza.

Lo que aprendimos trabajando con datos rurales

Diseñar un sistema de inventario para distribuidores de café, panela, cacao y miel del Huila nos puso frente a una realidad concreta: **los datos del mundo real no son uniformes**. Un lote de café pergamino de la Vereda El Paraíso tiene variedad, altitud y proceso de beneficio. Un lote de cacao de la Asociación Cacaotera del Macizo tiene días de fermentación y código de trazabilidad. Un frasco de miel tiene valor Brix y tipo de flora.

Intentar guardar todo eso en un esquema relacional rígido significaba elegir entre dos malos caminos: o creábamos decenas de columnas opcionales llenas de valores nulos, o creábamos tablas auxiliares que complicaban las consultas sin agregar valor real. Fue ahí cuando el modelo documental de MongoDB dejó de ser un concepto de clase y se convirtió en una solución concreta: el campo `datos_variables` absorbió toda esa heterogeneidad sin una sola migración.

Lo que nos enseñó el Teorema CAP

Antes del curso, la idea de que un sistema distribuido *no puede* garantizar consistencia, disponibilidad y tolerancia a particiones al mismo tiempo nos habría parecido una limitación académica sin consecuencias prácticas. Hoy entendemos por qué importa.

Cuando diseñamos los triggers de PostgreSQL para el descuento de stock, estábamos aplicando el modelo CP sin saberlo: preferimos consistencia fuerte aunque eso signifique lanzar una excepción y revertir toda la transacción. Un distribuidor de café no puede permitirse vender 999.999 kg que no existen. La integridad es innegociable.

Cuando diseñamos eventos `_historial` en MongoDB, aplicamos el modelo AP conscientemente: si un evento de auditoría tarda milisegundos en replicarse entre nodos, no pasa nada. La venta ya fue confirmada en PostgreSQL. El historial puede llegar después. Esa consistencia eventual no solo es aceptable, es la decisión correcta para un log de alta frecuencia de escritura.

Esa distinción, que antes era abstracta, ahora nos parece natural.

Los triggers como pensamiento declarativo

Uno de los cambios más importantes fue entender los triggers no como una característica avanzada de PostgreSQL, sino como una forma distinta de pensar la lógica de negocio. En lugar de decir "el código de la aplicación debe recordar descontar el stock después de cada venta", el trigger dice: "el descuento *es parte* de la inserción, siempre, sin excepción, independientemente de qué cliente o lenguaje acceda a la base de datos".

Eso es atomicidad en práctica, no en teoría. Y es una garantía que ninguna capa de aplicación puede reemplazar completamente.

Lo que llevamos hacia adelante

Este proyecto nos dejó dos preguntas que queremos seguir respondiendo:

1. **¿Cómo se garantiza la consistencia entre dos motores en producción?** El patrón Outbox con Kafka o RabbitMQ es la respuesta que identificamos, pero aún no hemos implementado. Ese es el siguiente paso real.
2. **¿Cómo se despliega esto en la nube?** PostgreSQL en Azure o AWS y MongoDB Atlas son las opciones naturales. Docker Compose ya es un paso en esa dirección.

Como ingenieros en formación, salimos de este semestre con algo más valioso que saber usar dos motores distintos: salimos sabiendo **cuándo usar cada uno y por qué**. Esa capacidad de justificar decisiones técnicas con criterios concretos —ACID cuando la integridad es innegociable, BASE cuando la flexibilidad y disponibilidad son prioritarias— es la diferencia entre ejecutar instrucciones y diseñar sistemas.